

# **Fire Weather Index Predictor**

(A Machine Learning Model to Predict Fire Weather Index)



## **Infosys Springboard Virtual Internship Program**

**Submitted by:**

Mansi Bambal

Under Guidance of Mentor Praveen

## **Project Statement:**

Wildfires pose a significant threat to ecosystems, human life, and property. The Fire Weather Index (FWI) is a crucial tool used by meteorological and environmental agencies worldwide to estimate wildfire potential. This project aims to build a machine learning model that predicts FWI based on real-time environmental data, enabling proactive wildfire risk management. The model is trained using Ridge Regression, deployed via a Flask web application, and supports early warning systems for wildfire hazards.

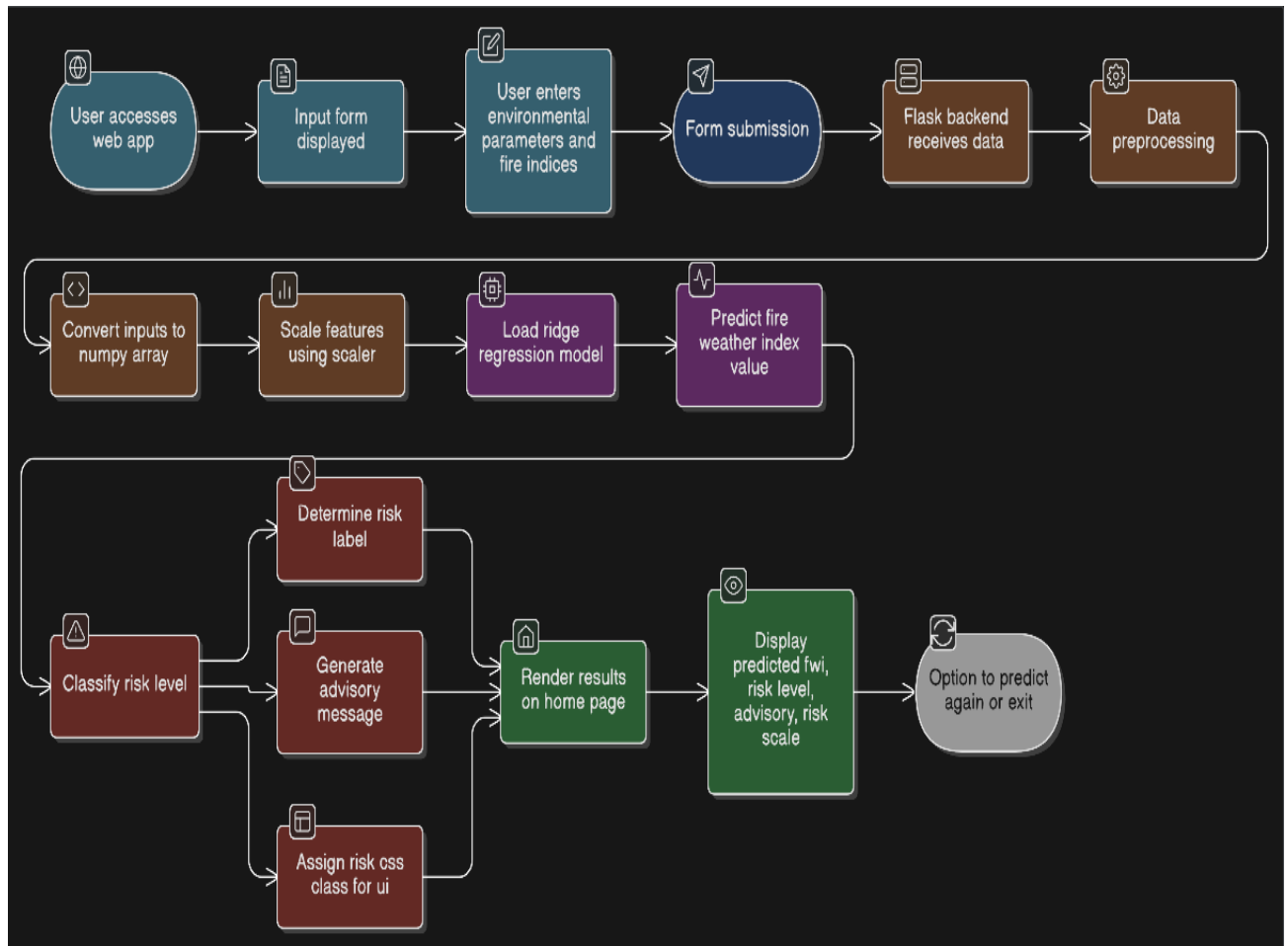
## **Outcomes:**

- A predictive ML model trained using Ridge Regression to forecast FWI.
- A pre-processing pipeline using StandardScaler for normalization.
- A Flask-based web app where users can input environmental values and get FWI predictions.
- A system that can help forest departments, emergency planners, and climate researchers make data driven decisions.

## **Modules to be implemented**

- Data Collection
- Data Exploration (EDA) and Data Preprocessing
- Feature Engineering and Scaling
- Model Training using Ridge Regression
- Evaluation and Optimization
- Deployment via Flask App
- Presentation and Documentation

## FWI Predictor System Workflow (Created using Mermaid):



### Explanation of the Diagram

- **Input:** User enters all environmental and fire indices through the web form.
- **Backend Processing:** Flask handles the request, preprocesses data (NumPy conversion + scaling), and loads the pre-trained Ridge model.
- **Prediction:** Model predicts FWI, which is then classified into a risk level with advisory.
- **Output:** home.html displays prediction, risk label, advisory, and a visual risk scale.

**Loop:** Users can choose to **predict again**, creating a cyclic flow.

# **Milestone 1**

## **Module 1: Data Collection**

### **Dataset Loading:**

- Loaded the raw dataset into a Pandas DataFrame for structured data handling and inspection.
- Verified data integrity and structure using:

```
df.head(), df.info()
```

- Confirmed the presence of relevant environmental features such as:
  - Temperature, Relative Humidity (RH), Wind Speed (Ws), Rain, FFMC, DMC, DC, ISI, BUI, and FWI (target variable).

### **Initial Mapping of Region:**

- Converted numeric region labels into meaningful names for clarity:
  - 0 → Bejaia
  - 1 → Sidi-Bel Abbes
- This improved readability during analysis and helped in understanding geographic behavior differences in fire conditions.

### **Typecasting Errors in Raw Data:**

- Encountered misalignment issues in the raw dataset (notably around row 167), where DC and FWI columns were stored as `object` type instead of numeric.
- Corrected by explicitly converting these columns back to float types to enable mathematical operations and model training.

### **Basic Data Checks:**

- Checked for missing values using:

```
df.isnull().sum()
```

to ensure completeness.

- Verified the absence of duplicate records using:

```
df.duplicated().sum()
```

ensuring no repeated data influenced results.

### Outcome:

- The dataset was successfully loaded, cleaned for type consistency, and made ready for exploration.
- Regional mapping and type correction laid a strong foundation for reliable preprocessing and modeling.

## Module 2: Data Exploration and Preprocessing

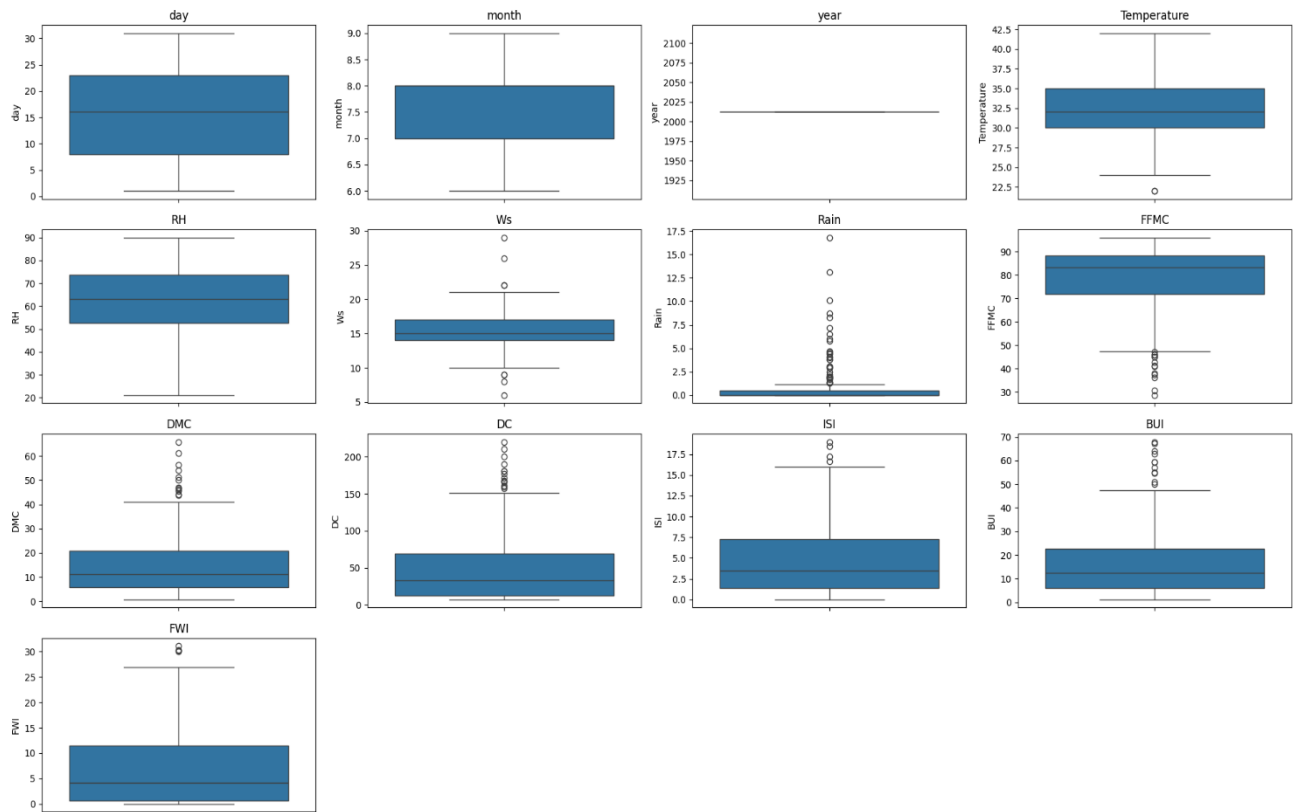
### Handling Missing Values:

- Trimmed spaces from column names using `df.columns.str.strip()` for consistency.
- For the **Classes** column, imputed missing values using the **mode**, ensuring data completeness without losing valuable samples.
- Remaining null values, if any, were dropped using:

```
df.dropna(inplace=True)
```

### Outlier Detection

- Used **boxplots** to identify outliers visually for all numerical features.
- Outliers were most notable in columns like **Rain**, **Ws**, and **fire indices** (FFMC, DMC, DC, ISI, BUI, FWD).



**Image:** The graphs illustrate a series of box plots representing the distribution of weather and fire danger indices such as Temperature, Relative Humidity (RH), Rain, Wind Speed (Ws), and fire indices (FFMC, DMC, DC, ISI, BUI, FWI). Each box summarizes the data spread, median, and interquartile range (IQR), while individual points outside the whiskers represent **outliers** — extreme observations beyond the normal range. Noticeable outliers are found particularly in **Rain**, **Wind Speed**, and multiple fire indices (**FFMC**, **DMC**, **DC**, **ISI**, **BUI**, **FWI**). These values may correspond to unusual weather events or exceptional fire conditions. The box plots help visually detect data irregularities, **skewed distributions**, and **inconsistent patterns** that might affect model training. This visualization was crucial for understanding feature behavior, as it highlighted which variables exhibit **natural variability** and which might require **scaling, transformation, or outlier handling** to ensure stable model performance later in the pipeline.

- The **IQR (Interquartile Range)** method was then applied programmatically to detect and understand statistical outliers.

### Dropping Features

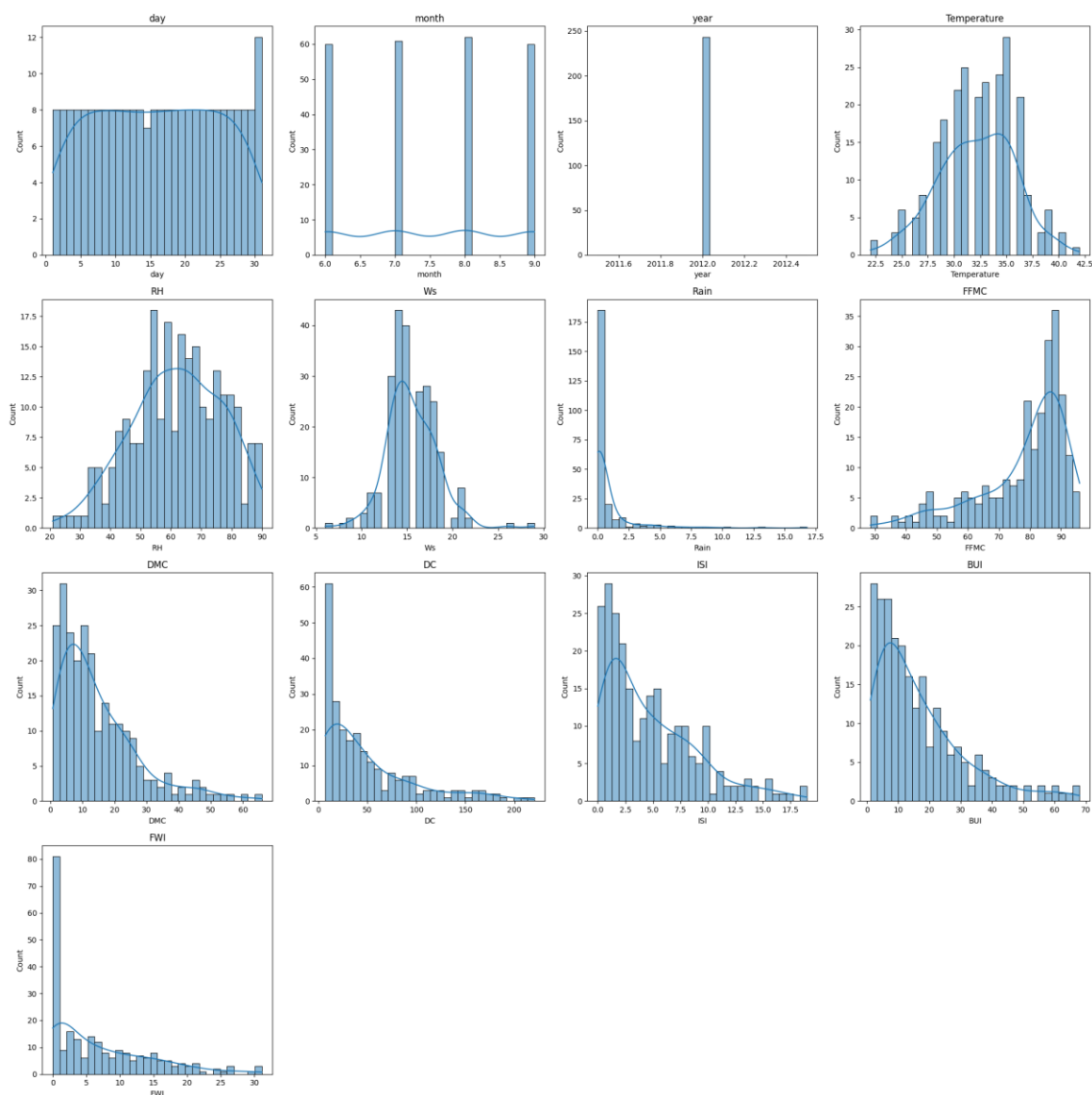
- Dropped the **Classes** column (since classification was not part of this regression task).
- Removed **day**, **month**, and **year** columns, as they didn't show significant correlation with FWI and could introduce noise.

## Distribution Visualization

- Plotted **histograms and density plots** to observe data spread and distribution shape.

### *Observations:*

- Most meteorological features like **Temperature, RH, and Ws** showed roughly normal distributions.
- Fire indices were **right-skewed**, showing that extreme fire conditions occur rarely but are crucial for prediction.



**Image:** The histograms and density plots display the distribution patterns of various environmental and fire-weather-related features. Variables such as **Temperature, Relative Humidity (RH), and Wind Speed (Ws)** show near-normal distributions, indicating that their

values are generally centered around the mean with moderate spread. **Time-based features** like day, month, and year appear **uniformly distributed**, reflecting balanced sampling across different periods. In contrast, **fire weather indices** such as **Rain, FFMC, DMC, DC, ISI, BUI, and FWI** exhibit **right-skewed distributions** — meaning most observations lie in lower ranges with a few extremely high values. These rare extremes likely represent **intense fire conditions or unusual weather events**, pulling the average upward. The **Region** feature is binary, showing two clear categories (0 and 1), corresponding to the two study areas — **Bejaia and Sidi-Bel Abbes**.

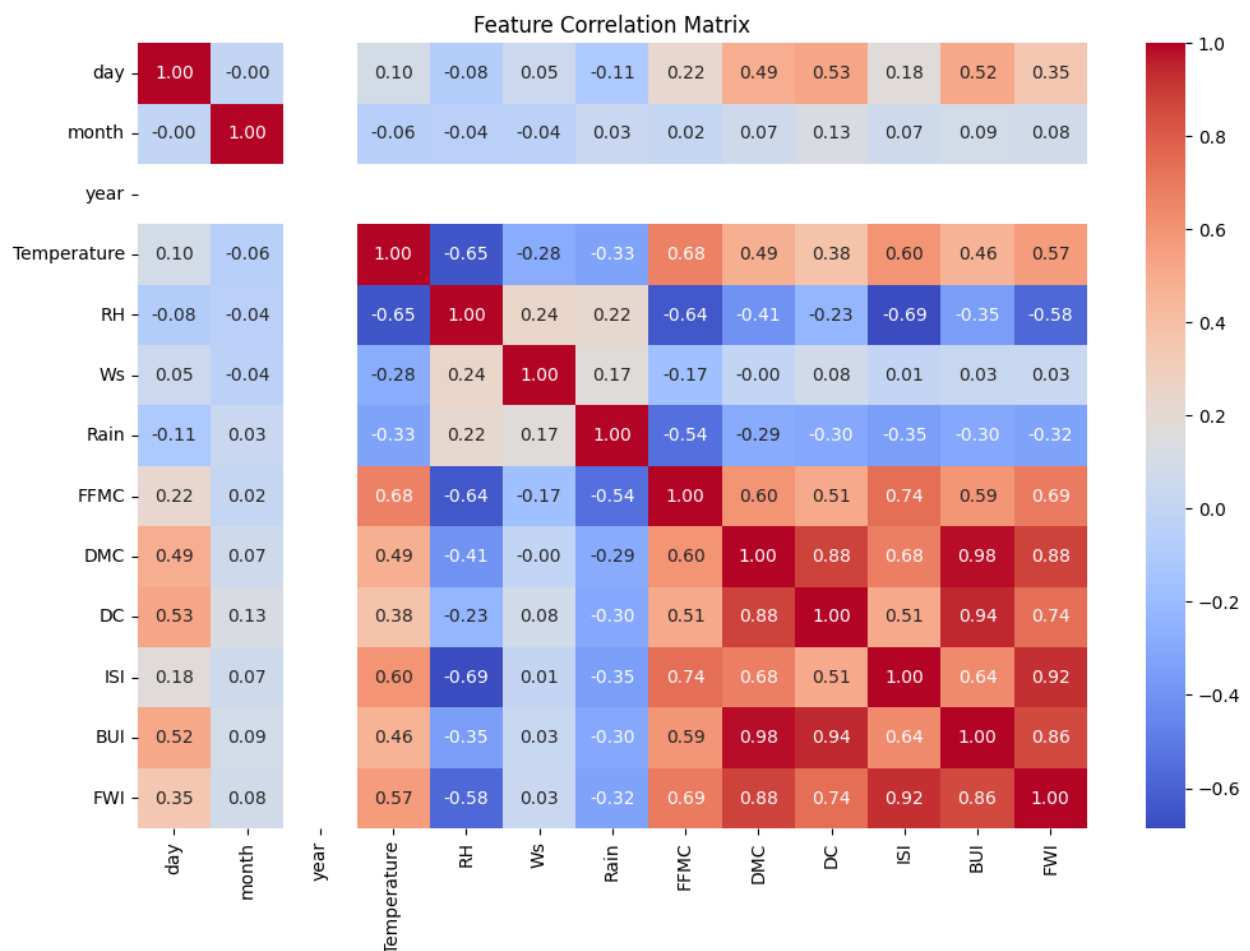
Overall, this visualization provided useful insights into how different features vary, helping identify potential scaling needs and ensuring that the dataset is well-understood before model training.

### Correlation Analysis

- Generated a **correlation heatmap** to identify relationships between variables.

#### Observed:

- Strong positive correlations among **DMC, DC, ISI, BUI, and FWI**.
- **Temperature** moderately correlated with FWI.
- **RH** negatively correlated, indicating higher humidity lowers fire intensity.
- These insights guided feature selection in Module 3.



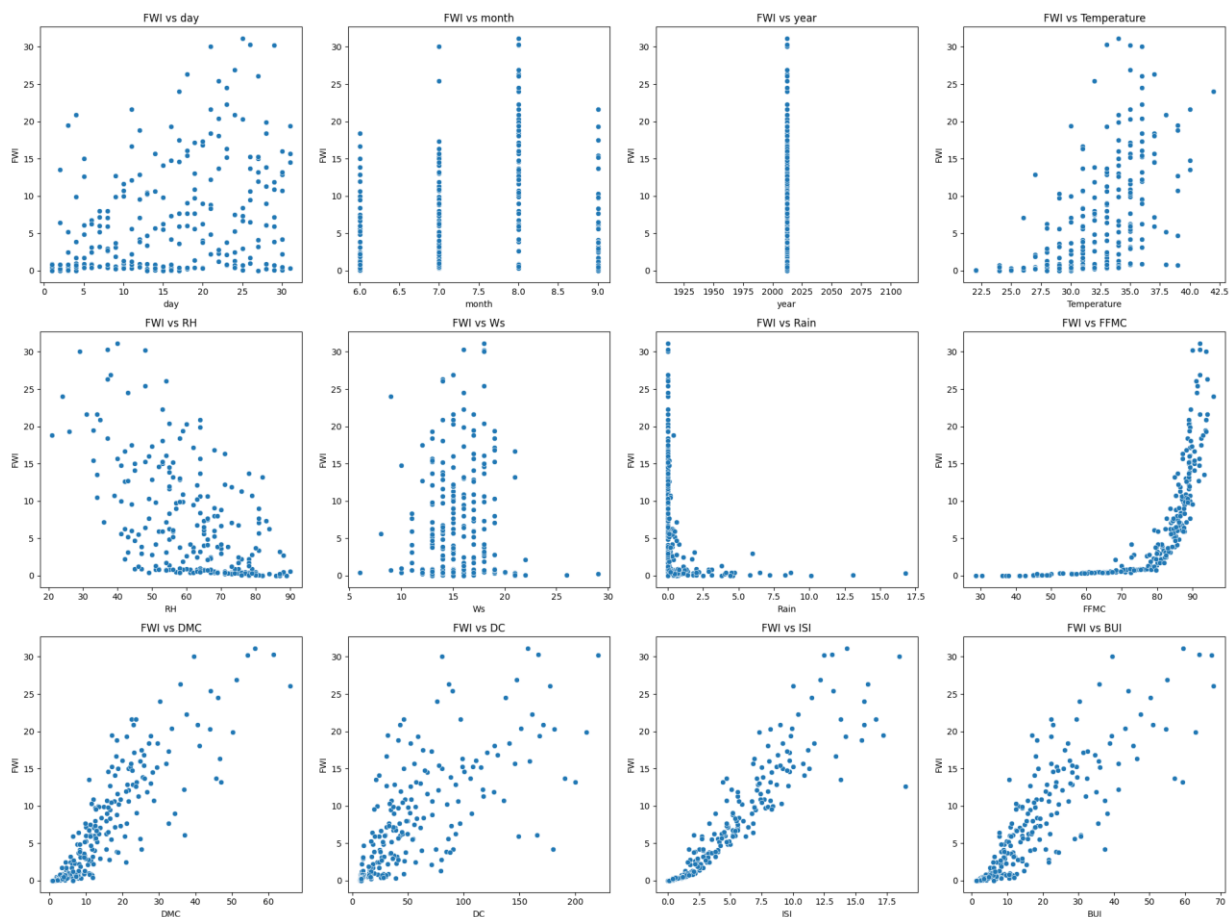
**Image:** This heatmap represents the **Feature Correlation Matrix**, which visually captures how strongly each variable is related to others. Correlation values range from **-1 to +1**, where values close to **+1 (deep red)** indicate a **strong positive correlation** — meaning both variables increase together — and values near **-1 (deep blue)** indicate a **strong negative correlation**, where one variable increases as the other decreases.

The analysis revealed **extremely high positive correlations** among fire-related indices such as **DMC, DC, ISI, BUI, and FWI**, confirming that these indices are closely linked and often rise under similar environmental conditions. **Temperature** also showed a **moderate positive correlation** with **FWI** and its sub-indices, suggesting that higher temperatures generally lead to greater fire intensity.

On the other hand, **Relative Humidity (RH)** displayed a **negative correlation** with most fire indices, which aligns with real-world patterns — higher humidity usually suppresses fire risk. This correlation matrix provided a solid foundation for **feature selection**, helping to retain only the most relevant predictors for modeling while avoiding redundancy due to multicollinearity.

## Scatter Plots

- Created scatter plots of each feature against FWI.
- Found nearly linear relationships between FWI and fire indices (e.g., DMC, FFMC, ISI), confirming their predictive importance.



**Image:** This figure presents a series of *scatter plots* illustrating how the **Fire Weather Index (FWI)** relates to each of the other environmental variables. The plots reveal a **strong positive and nearly linear relationship** between FWI and its sub-components — **DMC, ISI, BUI, and FFMC** — clearly confirming that these indices are the most influential predictors in determining fire intensity. As these variables increase, FWI also rises proportionally, indicating that they collectively capture similar underlying fire behavior conditions.

Additionally, **Temperature** shows a **clear positive correlation** with FWI— higher temperatures lead to higher fire danger levels — while **Relative Humidity (RH)** demonstrates a **weak to moderate negative relationship**, meaning that as humidity increases, FWI tends to decrease. This pattern aligns with environmental expectations since moisture in the air reduces the likelihood of fire spread.

A few points are scattered away from the main trend line, representing **minor outliers** or **extreme weather conditions**, but the overall pattern remains strong and consistent. These plots

*provided crucial visual confirmation that the selected features are meaningful and directly impact the target variable, FWI.*

### **Categorical Encoding**

- Converted the **Region** column (categorical) into numeric form using **Label Encoding** for model compatibility.

### **Saving Cleaned Data**

- After all transformations, the cleaned dataset was exported to a CSV file for further modeling:

```
df.to_csv("cleaned_dataset.csv", index=False)
```

### **Outcome:**

- The dataset was fully preprocessed — cleaned, outlier-checked, and statistically analyzed.
- The correlation and visualization stages provided strong evidence for selecting meaningful predictors for FWI modeling.

## **Milestone 2**

### **Module 3: Feature Engineering and Scaling**

#### **Feature Selection:**

- Identified the most relevant input features correlated with the target variable FWI (Fire Weather Index).
- Strongly correlated predictors included: Temperature, RH (Relative Humidity), Ws (Wind Speed), Rain, FFMC, DMC, DC, ISI, BUI, and Region.
- Features like day, month, year, and Classes were removed as they did not add predictive value and could introduce noise.
- This selection ensured a clean, focused dataset containing only the most influential variables.

#### **Data Splitting:**

- Split the dataset into:

- Input features (X) and Target variable (y = FWI).
- Used an 80:20 train-test split to train the model on a large portion of data while keeping a test set for unbiased evaluation.
- A fixed `random_state=42` was used for reproducibility.

### Feature Scaling:

- Applied StandardScaler to normalize features — essential since FWI predictors have different measurement units (e.g., temperature in °C, humidity in %, rain in mm).
- Scaling brought all features to a comparable range, improving model stability and training speed.
- The scaler was fitted on training data only and later applied to test data to avoid data leakage.

### Scaler Persistence:

- Saved the fitted scaler using pickle for consistent transformation during deployment:

```
with open("scaler.pkl", "wb") as f:
    pickle.dump(scaler, f)
```

- This ensures that the same scaling logic can be reused during prediction without retraining.

### Outcome:

- The processed data was now well-structured, scaled, and ready for regression modeling.
- This stage ensured model readiness and data consistency across future modules.

## **Module 4: Model Training using Ridge Regression**

### **Model Selection:**

- Multiple regression algorithms were tested to compare performance and select the most stable one:
  - Linear Regression: Standard regression model assuming a purely linear relationship.
  - Ridge Regression: Uses L2 regularization to reduce overfitting and handle multicollinearity among highly correlated fire indices.
  - Lasso Regression: Uses L1 regularization, performing both shrinkage and feature selection.
  - Random Forest Regressor: Ensemble-based model that captures non-linear patterns.
- Testing multiple models helped evaluate trade-offs between simplicity, interpretability, and predictive power.

### **Model Training:**

- Each model was trained on the scaled training dataset ( $X_{\text{train\_scaled}}$ ,  $y_{\text{train}}$ ).
- Scaling helped gradient-based models like Ridge and Lasso converge faster and avoid bias toward larger-magnitude features.

### **Model Evaluation:**

- Predictions were made on the test set, and models were compared using the  $R^2$  Score — a measure of how much variance in the target variable is explained by the model.
- $R^2 = 1.0$  means perfect prediction;  $R^2 = 0$  indicates no better than the mean.

### **Observed Results:**

- Linear Regression: 0.988
- Ridge Regression: 0.982
- Lasso Regression: 0.987

- Random Forest: 0.978

### **Model Comparison and Insights:**

- Linear Regression achieved the highest  $R^2$ , but may slightly overfit.
- Ridge Regression provided excellent generalization and effectively handled correlated predictors (like DMC, DC, and BUI).
- Lasso Regression performed nearly as well and could be beneficial for feature selection in further tuning.
- Random Forest had slightly lower performance, possibly due to the dataset size and linear dominance of relationships.

### **Model Saving:**

- Based on balance between stability and interpretability, Ridge Regression was finalized.
- Saved the trained model for later deployment:

```
with open("ridge.pkl", "wb") as f:  
    pickle.dump(ridge_model, f)
```

### **Outcome:**

- Ridge Regression achieved strong predictive capability with minimal overfitting.
- Final  $R^2$  Score: 0.982
- The model was stored for further evaluation and optimization.

## **Milestone 3**

### **Module 5: Evaluation and Optimization**

#### **Model Evaluation Metrics:**

- The Ridge model was further evaluated using multiple statistical metrics:
  - Mean Absolute Error (MAE): 0.58
    - Represents the average absolute difference between predicted and actual FWI values.
    - Easier to interpret as it shows direct prediction deviation in FWI units.
  - Root Mean Squared Error (RMSE): 0.74
    - Measures the square root of average squared errors.
    - Larger errors are penalized more, providing a better measure of overall model accuracy.
  - R<sup>2</sup> Score: 0.98
    - Indicates that 98% of the variability in FWI is explained by the model — a sign of excellent predictive power.

#### **Visual Analysis:**

- A Predicted vs Actual scatter plot was drawn to visually assess model performance.
- Points were closely aligned along the 45° line, showing that predictions were very close to true FWI values.
- This confirmed that the Ridge Regression model effectively captured the underlying relationships between environmental features and FWI.

#### **Parameter Optimization:**

- Conducted Grid Search to fine-tune the regularization parameter **alpha** for Ridge Regression.
- The best parameter was found to be  $\alpha = 1$ , balancing bias and variance optimally.

- Since this matched the training configuration, no retraining was necessary.

### **Insights:**

- The model generalizes extremely well with no signs of underfitting or overfitting.
- Both numerical and visual evaluations confirm that Ridge Regression is reliable for predicting Fire Weather Index in similar datasets.

### **Outcome:**

- Finalized Ridge model with optimal hyperparameters and validated performance.
- Model metrics demonstrated robustness and precision, ready for real-world integration or deployment.

## **Milestone 4**

### **Module 6: Development via Flask App**

#### **Flask App Structure:**

The Flask application follows a standard structure:

- **app.py** – Main Python file that contains the Flask app, routes, input processing, model loading, and prediction logic.
- **ridge.pkl & scaler.pkl** – Serialized machine learning model and feature scaler used for prediction.
- **templates/** – Folder containing HTML files:
  - index.html → User input form.
  - home.html → Displays prediction results.
- **static/** – Folder containing CSS (style.css) and other static resources for styling and UI.

This structure separates application logic, templates, and static assets for better organization and maintainability.

## App Workflow and Key Code:

### Step 1: Load Model and Scaler

At app startup, the pre-trained model and scaler are loaded:

```
with open("scaler.pkl", "rb") as f:
    scaler = pickle.load(f)

with open("ridge.pkl", "rb") as f:
    model = pickle.load(f)
```

- **Scaler:** Normalizes input features to match the training data.
- **Model:** Ridge Regression model predicts the Fire Weather Index (FWI).

### Step 2: Define Risk Categorization

The `fwi_to_risk` function maps the predicted Fire Weather Index (FWI) value to a **risk category** and provides a corresponding **safety advisory**. It also assigns a **CSS class name** for UI styling.

#### **Logic:**

1. **Low Risk ( $FWI \leq 5$ ):**
  - Fire risk is minimal.
  - Advisory: "Conditions are safe — fire risk is minimal."
  - UI class: "low"
2. **Moderate Risk ( $5 < FWI \leq 10$ ):**
  - Fires may start under dry conditions.
  - Advisory: "Be alert — fires can start under dry conditions."
  - UI class: "moderate"
3. **High Risk ( $10 < FWI \leq 20$ ):**
  - Fire spread is likely under current conditions.
  - Advisory: "Caution! Fire spread is likely under current conditions."
  - UI class: "high"

#### 4. Extreme Risk (FWI > 20):

- Very high chance of fire ignition and rapid spread.
- Advisory: "Danger! High chance of fire ignition and rapid spread."
- UI class: "extreme"
- Returns a **risk label**, **advisory message**, and **CSS class** for UI styling.

### Step 3: Flask Routes and Prediction

#### Index Route

```
@app.route("/", methods=["GET"])  
  
def index():  
    return render_template("index.html")
```

- Displays the input form to the user.

#### Prediction Route

```
@app.route("/predict", methods=["POST"])
```

- Handles form submission, preprocessing, prediction, and risk categorization.
- Returns the results dynamically to **home.html**.

### Step 4: HTML Templates

#### index.html:

- Form fields include environmental parameters (Temperature, RH, Ws, Rain) and fire indices (FFMC, DMC, DC, ISI, BUI) along with a **region dropdown**.
- Submits POST request to /predict.

#### home.html:

- Displays:
  - Predicted FWI value
  - Risk level (Low/Moderate/High/Extreme)
  - Safety advisory
  - FWI scale for reference

- Includes a “**Predict again**” button to return to input form.

### **Step 5: CSS and Styling**

- Dark-themed, gradient-based UI with **responsive design**.
- Color-coded risk levels:
  - Low → Blue
  - Moderate → Orange
  - High → Dark Orange
  - Extreme → Red with pulsing glow
- Animations for risk bars and interactive UI.

### **Step 6: Running the App**

*python app.py*

- Open `http://127.0.0.1:5000/` in a browser.
- Fill the form and submit to see the prediction and risk assessment.

### **Outcomes:**

- The app now provides **enhanced user experience** with visually appealing results.
- Predicted FWI values are displayed along with **color-coded risk levels** and **safety advisories**.
- Users can easily interpret fire risk through the **FWI scale reference**.
- The interface is **responsive**, making it accessible on both desktop and mobile devices.
- Overall, Module 7 makes the FWI prediction system **more informative, interactive, and user-friendly**.

## **Module 7: Presentation and Documentation**

### **Project Summary:**

- The FWI Predictor is a Flask-based web application that predicts the Fire Weather Index (FWI) using environmental parameters and fire weather indices.
- Users can input temperature, humidity, wind speed, rainfall, and other fire-related indices like FFMC, DMC, DC, ISI, BUI, FWI, and the app provides:
  - A predicted FWI value
  - Risk level classification (Low, Moderate, High, Extreme)
  - Safety advisory messages
  - Visual indicators for better understanding

Purpose: Provides actionable insights for forest fire prevention and monitoring.

### **System Architecture:**

- The application follows a client-server model:
  1. Frontend (Client):
    - HTML, CSS, and form inputs for environmental data.
    - Displays results with dynamic styling and visual risk indicators.
  2. Backend (Server):
    - Flask app handles form data, scaling, and prediction using the pre-trained Ridge Regression model.
    - Returns results to the frontend along with risk classification.
  3. Data Processing:
    - Input features are scaled using StandardScaler before prediction.
    - Predicted FWI is mapped to risk labels and advisories.

### **Workflow and End-to-End Pipeline:**

The end-to-end process of the FWI Predictor can be described as:

1. User Input:
  - User enters environmental parameters and fire indices in the web form.
2. Data Preprocessing:
  - Input values are collected via Flask's request.form.
  - Features are converted into a NumPy array and scaled using the loaded scaler.pkl.
3. Prediction:
  - Scaled inputs are passed to the Ridge Regression model (ridge.pkl).
  - Model predicts the FWI value.
4. Risk Classification:
  - The predicted FWI is passed to fwi\_to\_risk function.
  - Returns risk label, advisory message, and CSS class for UI display.
5. Result Display:
  - Results are rendered on home.html with:
    - Predicted FWI value
    - Color-coded risk level card
    - Safety advisory
    - FWI scale reference
  - Users can predict again using the reset button.

### **Web App Screenshots:**

- Input Page (index.html):
  - Contains fields for all environmental parameters.
  - Submit button triggers prediction.
  - Clean layout with interactive form fields.
- Result Page (home.html):
  - Displays predicted FWI value.
  - Shows risk level with color-coded card and animated risk bar.
  - Advisory section highlights safety measures.
  - Reference FWI scale helps users interpret results.



## Fire Weather Index Predictor

Advanced AI-powered fire risk assessment system

---

 **Environmental Parameters**

---

Temperature (°C)

32.5

Relative Humidity (%)

60

Wind Speed (km/h)

15

Rainfall (mm)

0.0

 **Fire Weather Indices**

---

FFMC ⓘ

85.4

DMC ⓘ

10.5

DC ⓘ

30.2

ISI ⓘ

5.6

BUI ⓘ

12.3

Region

Select region ▼

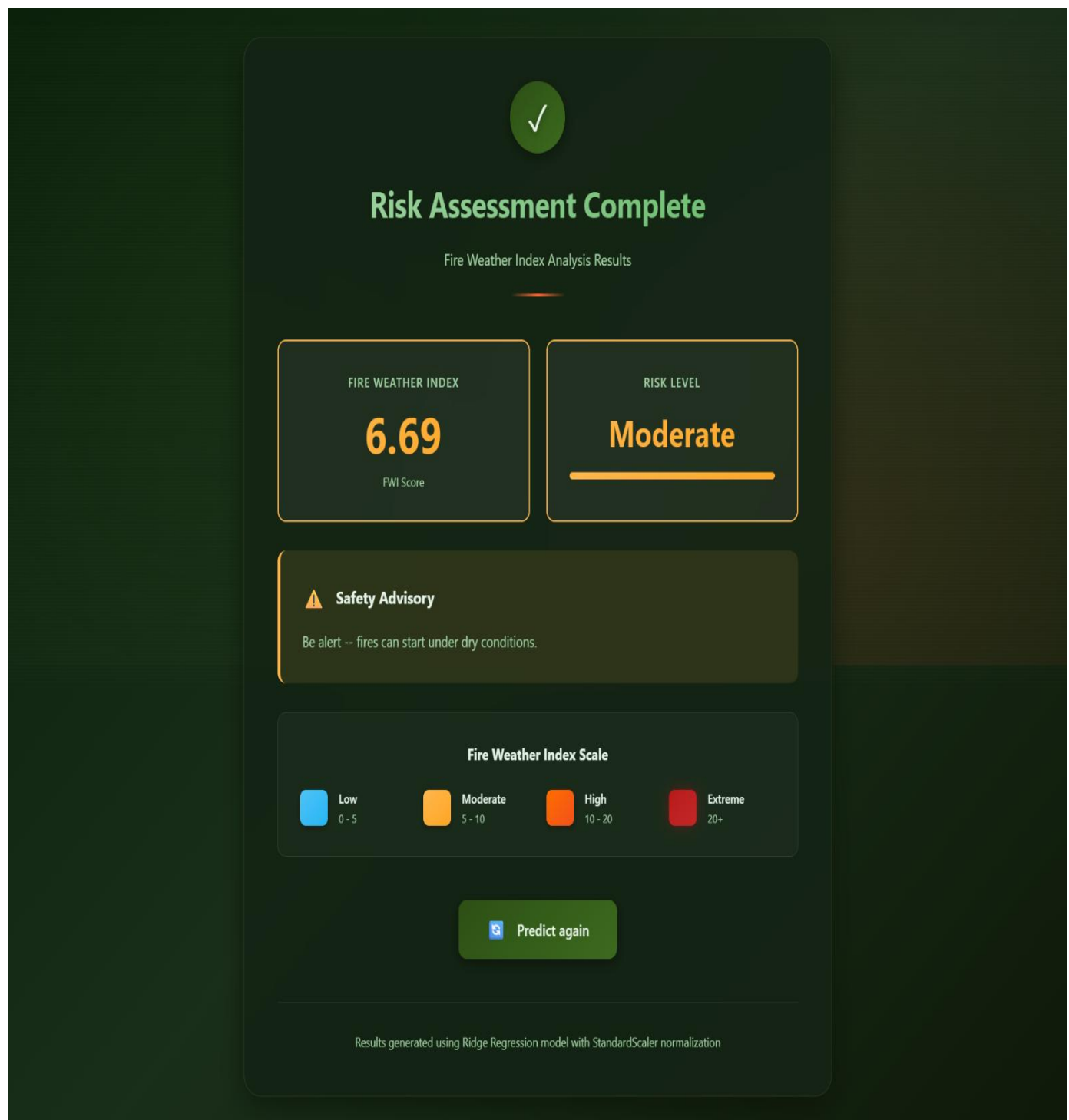


RIDGE REGRESSION MODEL

STANDARDSCALER NORMALIZATION

Providing accurate fire weather predictions for forest safety

**Image 1:** Input Interface Screenshot: This image shows the user interface of the Fire Weather Index (FWI) Predictor where users can enter environmental parameters like temperature, relative humidity, wind speed, and rainfall. It also displays fields for fire weather indices (FFMC, DMC, DC, ISI, BUI) and a region selection dropdown. A clear “Calculate Fire Risk” button initiates the prediction. Additional labels indicate the use of Ridge Regression and StandardScaler normalization for accurate predictions.



**Image 2:** This image presents the results screen of the fire risk assessment, titled "Risk Assessment Complete." A green checkmark at the top signifies a successful operation. The core results are displayed in two main panels: the Fire Weather Index Score is 6.69, and the corresponding Risk Level is Moderate. Below the results, a yellow-boxed "Safety Advisory" warns: "Be alert -- fires can start under dry conditions." A Fire Weather Index Scale is clearly mapped out with color codes: Blue for Low (0-5), Orange for Moderate (5-10), Red-Orange for High (10-20), and Dark Red for Extreme (20+). Finally, a green button prompts the user

to "Predict again," and a footer notes that the "Results [were] generated using Ridge Regression model with StandardScaler normalization."

*This output is for the following input values:*

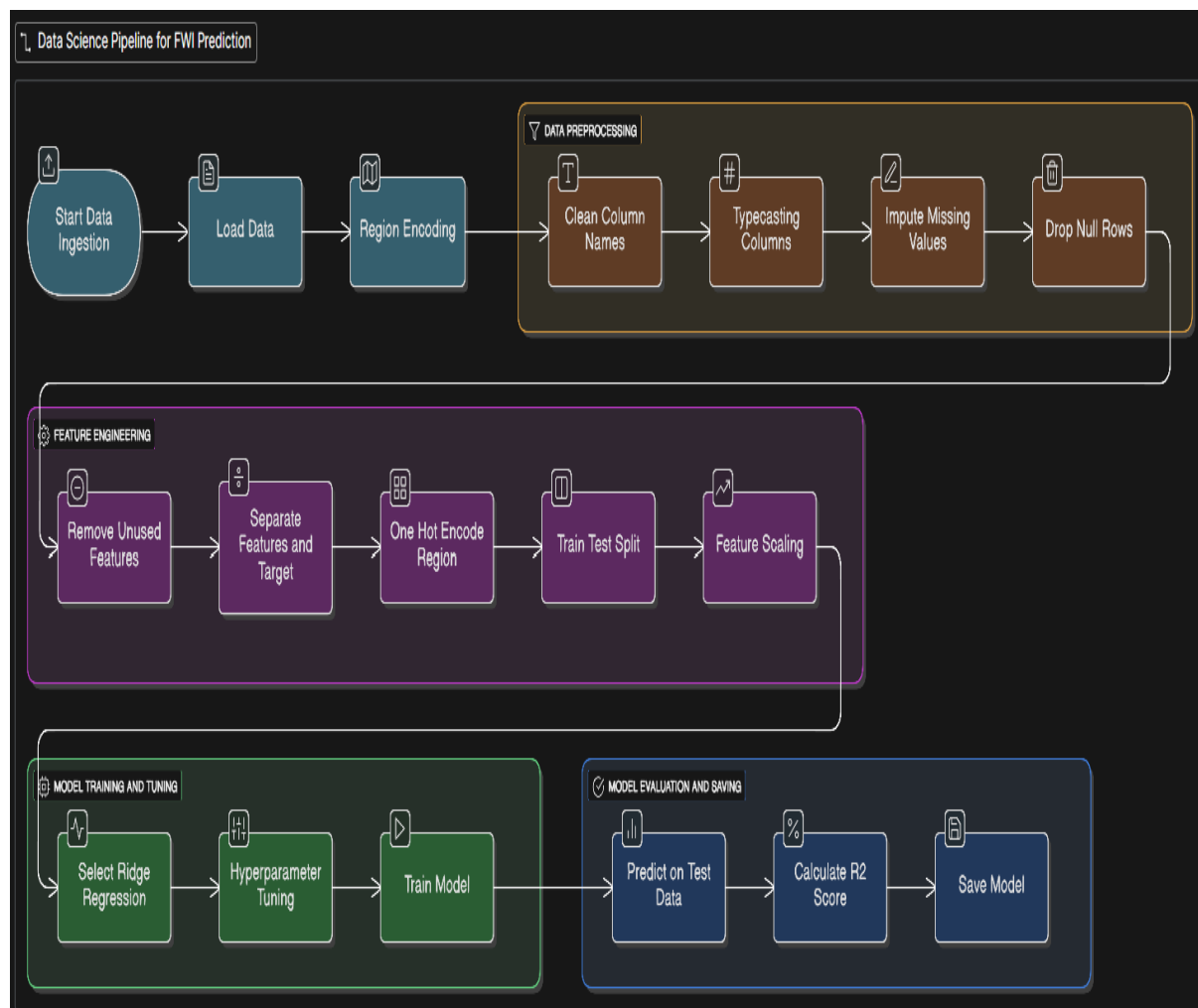
***Environmental Parameters:***

- *Temperature: 32.5°C*
- *Relative Humidity: 60%*
- *Wind Speed: 15 km/h*
- *Rainfall: 0.0 mm*

***Fire Weather Indices:***

- *FFMC: 85.4*
- *DMC: 10.5*
- *DC: 30.2*
- *ISI: 5.6*
- *BUI: 12.3*
- *Region: Bejaia*

## FWI Predictor Model: End-to-End ML Pipeline:



**Image:** The image illustrates a five-stage Machine Learning pipeline for an FWI (Fire Weather Index) Predictor Model, using Ridge Regression.

The pipeline's flow is:

1. *Data Ingestion/Preprocessing:* Data is loaded, cleaned (nulls dropped, columns renamed), and types are converted.
2. *Feature Engineering:* Unused columns are removed, the target (FWI) is separated, the 'Region' feature is One-Hot Encoded, and the dataset is split into Training and Test sets before Feature Scaling (Standardization).
3. *Model Training/Tuning:* A Ridge Regression model is selected, its hyperparameters are optimized using GridSearchCV, and the model is trained.
4. *Evaluation/Saving:* Predictions are made on the test data, the R2 Score is calculated for performance, and the final trained model is saved (e.g., using pickle).

### **File Organization:**

- app.py: Main Flask application script handling routes, prediction, and preprocessing.
- templates/
  - index.html – Input form
  - home.html – Results page
- static/
  - style.css – Styling for frontend
- scaler.pkl – Pre-trained StandardScaler object
- ridge.pkl – Pre-trained Ridge Regression model
- README.md / Documentation – Project summary, instructions, and workflow diagrams

Proper organization ensures easy review, maintenance, and GitHub publishing.

### **Outcome:**

- Complete documentation prepared, covering project summary, architecture, and workflow.
- Screenshots and results clearly demonstrate input-to-output pipeline.
- System architecture and workflow diagrams were created using, providing a visual representation of the app flow.
- All files are organized for submission, review, and GitHub publication, making the project professional and presentable.
- Overall, Module 7 ensures the project is well-documented, easy to understand, and visually supported for reviewers.